

CS 611: Program Analysis

Course Code	CS 611
Course Name	Program Analysis
L-T-P-C	3-1-0-4
Prerequisites	CS202: Data Structures and Algorithms or equivalent; CS208: Mathematical Foundations of Computer Science or equivalent; CS304: Formal Languages and Automata Theory or equivalent
Intended for	B.Tech., M.Tech., MS, PhD
Distribution	Discipline Elective for BTech CSE, Elective for others
Approval	37th BoA (Oct 2020); 30th Senate (Feb 2021), in the M.Tech Computer Science and Engineering curriculum

Course Contents

- **Introduction to static analysis:** Concrete versus abstract semantics. Abstract interpretation. Galois connection. Symbolic execution. Control-flow graphs. Iterative data flow analysis. Lattices and monotonicity. Analysis dimensions. [10 Lectures]
- **Heap analysis. Heap modeling:** Points-to information. Andersen's and Steensgaard's pointer analyses. Variations: alias analysis, null-check analysis, escape analysis. [9 Lectures]
- **Interprocedural analysis:** Call-graph construction. Context sensitivity: functional and call-string approaches. Various context abstractions: value and lsrvt contexts, object and type sensitivity. Heap cloning. [10 Lectures]
- **Strategies for efficiency:** Demand-driven analysis. Program slicing. Analysis staging. Partial analysis. Efficient data structures. Heuristics and machine learning. [9 Lectures]
- **Language features and challenges:** Lexical and dynamic scoping. Eager and lazy evaluation. Call-backs and reflection. Concurrency and synchronization: may-happen-in-parallel analysis. Speculative optimizations and deoptimization. Dynamic analysis. [10 Lectures]
- **Sneak peek into applications:** Type checking. Bug detection. Program correctness. Program synthesis and repair. Software refactoring. [8 Lectures]

Tutorials and assignments

This course involves hands-on practice with writing different program analyses, implementing the techniques for efficiency, and learning various associated tools and language features. The classes will teach the theory, and tutorials would train the students on the various skill-sets required to finish take home programming assignments.

It may be noted that as the course covers several recent topics related to designing analyses that are precise-yet-efficient, the classes will use one or two important analyses as running examples (examples being alias analysis and pointer analysis for resolving virtual calls). Hence, another aim of the tutorial hours would be to help students imbibe the concepts learnt by making them write specifications of different analyses and optimization strategies.

The take-home assignments will be based on implementing and understanding (a) intra- and interprocedural analyses; (b) strategies for imparting efficiency; (c) examples of handling tricky language features; and (d) non-trivial applications such as parallelization, refactoring and security.

References

1. Flemming Nielson, Hanne Riis Nielson and Chris Hankin, **Principles of Program Analysis**, Corrected Edition, Springer, 1999.
2. Uday P. Khedker, Amitabha Sanyal and Bageshri Karkare, **Data Flow Analysis: Theory and Practice**, CRC Press, 2009.
3. Y. N. Srikant and Priti Shankar, **The Compiler Design Handbook**, 2nd Edition, CRC Press, 2007.
4. Various research papers related to the course content.